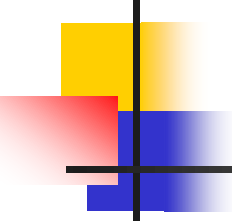


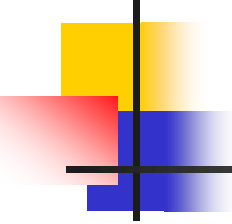
Database Systems and Data-Driven Applications

Transactions and Concurrency
Control – Part 1



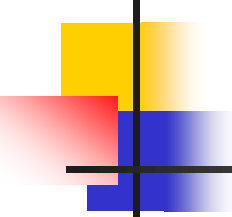
Last Week...

- Query Optimization Overview
- Any questions?



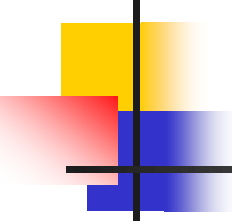
This Lecture...

- Transactions, and
- Concurrency Control



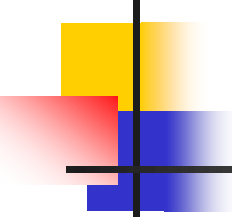
Transactions...

- A user's program may carry out many operations on the data retrieved from the database, but the DBMS is only concerned about what data is read/written from/to the database.
- A transaction is the DBMS's abstract view of a user program: a sequence of reads and writes.
 - Example: Transferring a balance from Account A to Account B



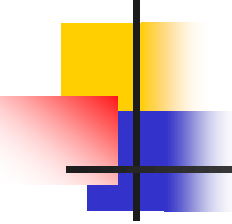
Transactions... (contd.)

- Properties of a transaction...
 - **A**tomicity
 - **C**onsistency
 - **I**solation
 - **D**urability



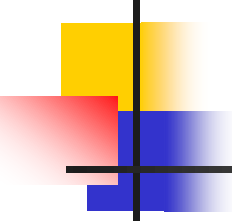
Atomicity of Transactions

- This property says that the transaction is executed as a single unit of work. That is, for every transaction, either all actions within the transaction are carried out or none are.
- A transaction might *commit* after completing all its actions, or it could *abort* (or be aborted by the DBMS) after executing some actions.
- In a committed transaction all actions are executed within a DBMS and an aborted transaction none of the actions are executed
 - DBMS *logs* all actions so that it can *undo* the actions of aborted transactions.



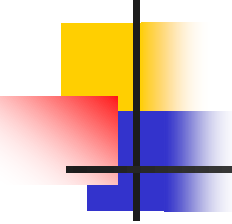
Consistency of Transactions

- **Database consistency** is the property that every transaction sees a consistent database instance.
- Users are mainly responsible for ensuring transaction consistency.
- ICs are verified by DBMSs. But program logic must guarantee the consistency of a transactions
 - E.g. transferring the balance from one account to another. The total amount in both accounts before and after the transaction must be equal.



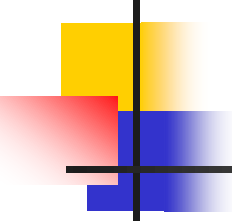
Isolation of Transactions

- The isolation property ensures that even though actions of several transactions are interleaved, the net effect is identical to executing all transactions in some serial order.
- Example



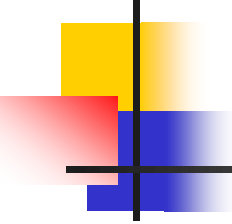
Durability of Transactions

- This property ensures that transactions survive system crashes and failures.
- That is, a committed transaction is always reflected in the database & an uncommitted transaction is always rolled back after a system crash
- DBMSs maintains a log & recovery manager is responsible to ensuring atomicity and durability properties



Transactions & Schedules...

- A transaction is seen by a DBMS as a series of ordered **actions**
- **Actions** include **reads** & **writes** of database objects
- Notation:
 - Transaction T reading object O: $R_T(O)$
 - Transaction T writing object O: $W_T(O)$

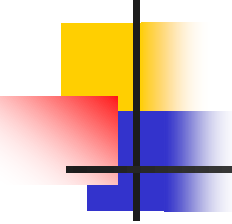


Transactions & Schedules...

(contd.)

- In addition, the final action of a transaction is *commit* or *abort*.
- Notation:
 - Committing transaction T: Commit_T
 - Aborting transaction T: Abort_T

Note: We omit the subscript T where unambiguous



Transactions & Schedules...

(contd.)

- **Schedule:** A list of actions (*reads, writes, commits, aborts*) from a set of transactions & the order of actions in the schedule is the same as the order of actions in transactions
- Intuitively, a schedule represents an actual or potential execution sequence.

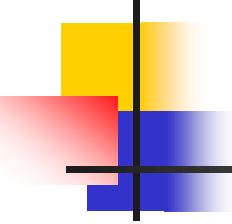
Transactions & Schedules...

(contd.)

- Example

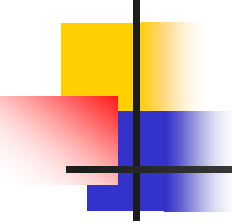


$T1$	$T2$
$R(A)$	
$W(A)$	
	$R(B)$
	$W(B)$
$R(C)$	
$W(C)$	



Concurrent Execution of Transactions

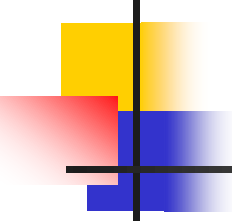
- DBMS interleaves actions of different transactions to improve performance
- Motivation for concurrent transactions
 - CPU can process one transaction while another is waiting for a page to be read from disk
 - Interleaving short transactions with longer transactions allows to complete quicker



Concurrent Execution of Transactions... (contd.)

- Example

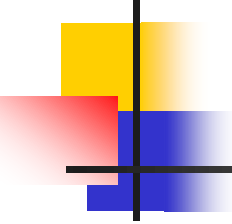
- T1: A transaction calculates the interest for all accounts in the bank (2 hours)
- T2: A customer wants to withdraw funds from the account



Scheduling Transactions

- *Serial schedule:* Schedule that does not interleave the actions of different transactions.
- *Equivalent schedules:* For any database state, the effect (on the set of objects in the database) of executing the first schedule is identical to the effect of executing the second schedule.
- *Serializable schedule:* A schedule that is equivalent to some serial execution of the transactions.

(Note: If each transaction preserves consistency, every serializable schedule preserves consistency.)

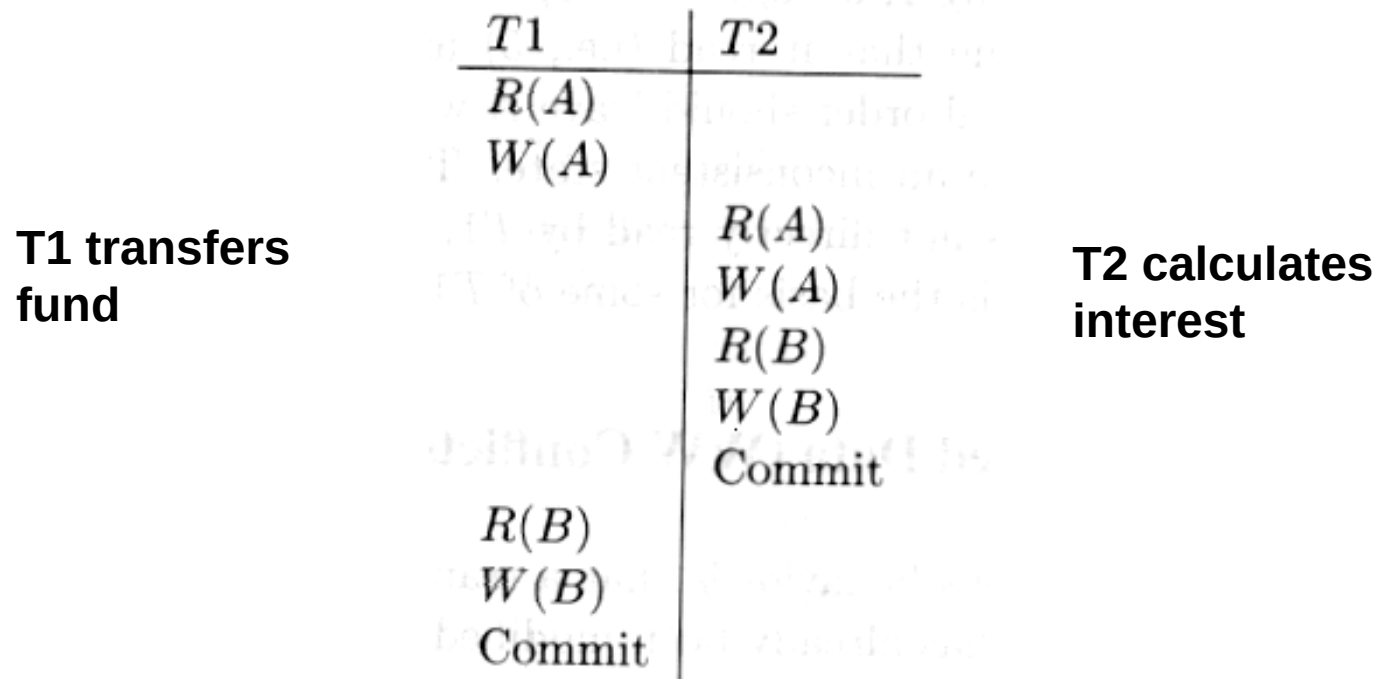


Serializability

- Multiple transactions executed concurrently must be equivalent to some serial execution of the transactions in order to preserve consistency
- * Order of transactions within the schedule doesn't matter

Anomalies with Interleaved Execution

- Reading Uncommitted Data (WR Conflicts, “dirty reads”):



Anomalies with Interleaved Execution... (contd.)

- Unrepeatable Reads (RW Conflicts):

T1:	R(A),	R(A), W(A), C
T2:	R(A), W(A), C	

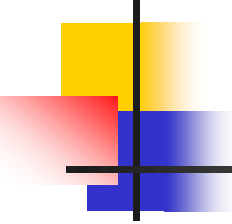
Example T1 is incrementing A by 1 and T2 is decrementing A by 1

Anomalies with Interleaved Execution... (contd.)

- Overwriting Uncommitted Data (WW Conflicts):

T1:	W(A),	W(B), C
T2:	W(A), W(B), C	

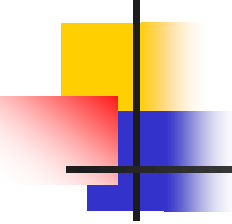
**Values of A and B are equal.
T1 makes A & B to be 1000.
T2 makes A & B to be 2000.**



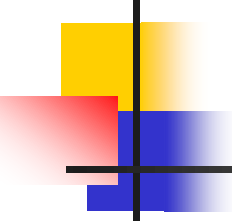
Unserializability

- The root cause for unserializable schedules (or violation of *Isolation* property) are conflicts.
 - WR conflict
 - RW conflict
 - WW conflict
- Idea: Avoid conflicts!!!

Lock-Based Concurrency Control

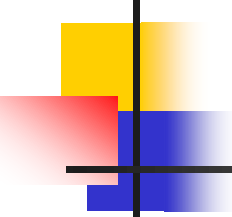


- *Strict Two-phase Locking (Strict 2PL) Protocol:*
 - Each Xact must obtain a *S (shared)* lock on object before reading, and an *X (exclusive)* lock on object before writing.
 - All locks held by a transaction are released when the transaction completes
 - If an Xact holds an X lock on an object, no other Xact can get a lock (S or X) on that object.



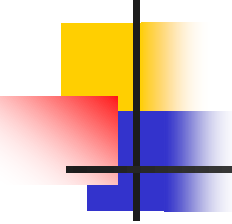
Strict 2PL

- Strict 2PL does not allow
 - WR conflict
 - RW conflict, or
 - WW conflict
- How?
- Therefore, Strict 2PL allows only serializable schedules.



Aborting a Transaction

- If a transaction T_i is aborted, all its actions have to be undone. Not only that, if T_j reads an object last written by T_i , T_j must be aborted as well!
- Some considerations with aborts:
 - Cascading aborts
 - Unrecoverable Schedules



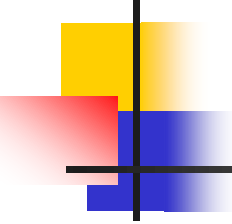
Cascading Abort

- Abort of one transaction causes the abort of another transaction

- Example

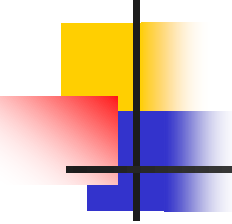
T_1	T_2
$W(A)$	
abort	$R(A)$

- T_1 's abort causes abort of T_2 as well.



Cascading abort (contd.)

- Root cause for cascading abort:
 - WR conflict +
 - Transaction which wrote the data aborts
- Good News: Strict 2PL does not allow cascading aborts!!!
 - How?
- This is also known as ***avoiding cascading aborts***

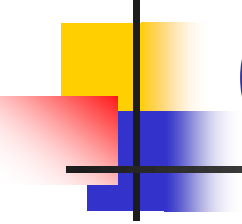


Unrecoverable Schedules

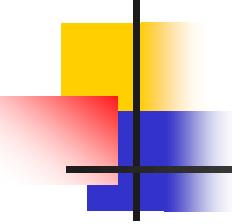
- An unrecoverable schedule:

T₁	T₂
W(A)	
	R(A)
	W(B)
	Commit
abort	

Unrecoverable Schedules (contd.)

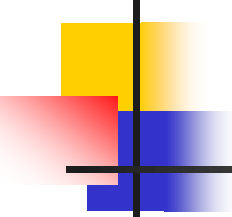


- Root cause for unrecoverable schedule:
 - WR conflict +
 - Transaction which read the data commits before transaction which wrote the data
- Good News: Strict 2PL does not allow unrecoverable schedules!!!
 - How?



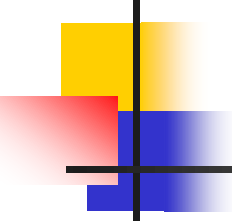
The Log

- In order to *undo* the actions of an aborted transaction, the DBMS maintains a *log* in which every write is recorded. This mechanism is also used to recover from system crashes: all active Xacts at the time of the crash are aborted when the system comes back up



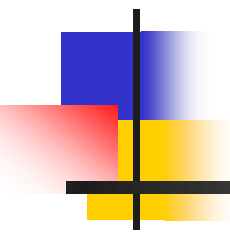
The Log (contd.)

- The following actions are recorded in the log:
 - *Ti writes an object:* the old value and the new value.
 - Log record must go to disk before the changed page!
 - *Ti commits/aborts:* a log record indicating this action.
- Log records are chained together by Xact id, so it's easy to undo a specific Xact.
- Log is often *duplexed* and *archived* on stable storage.
- All log related activities (and in fact, all CC related activities such as lock/unlock, dealing with deadlocks etc.) are handled transparently by the DBMS.



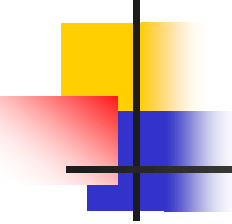
Summary

- Transaction
- Properties
- Interleaved Execution
- Conflicts
- Avoiding conflicts with Strict 2PL Protocol
- Aborting Transaction and Log



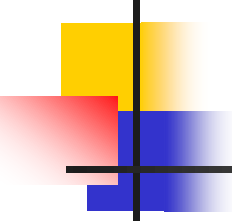
Database Systems and Data-Driven Applications

Transactions and Concurrency
Control – Part 2



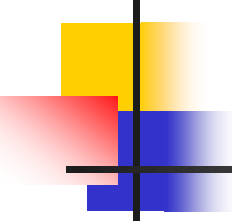
Overheads of Lock-based CC Protocols

- Locking based CC Protocols comes with a numbers of overheads:
 - Maintaining and handling locks
 - Handling deadlocks
 - Handling phantoms



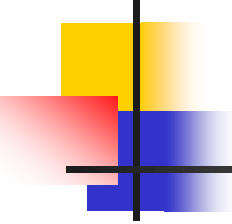
Deadlocks

- Deadlock: Cycle of transactions waiting for locks to be released by each other.
- Two ways of dealing with deadlocks:
 - Deadlock prevention
 - Deadlock detection



Deadlock Prevention

- Assign priorities based on timestamps. Assume T_i wants a lock that T_j holds. Two policies are possible:
 - Wait-Die: If T_i has higher priority, T_i waits for T_j ; otherwise T_i aborts
 - Wound-wait: If T_i has higher priority, T_j aborts; otherwise T_i waits
- If a transaction re-starts, make sure it has its original timestamp.
 - Why?



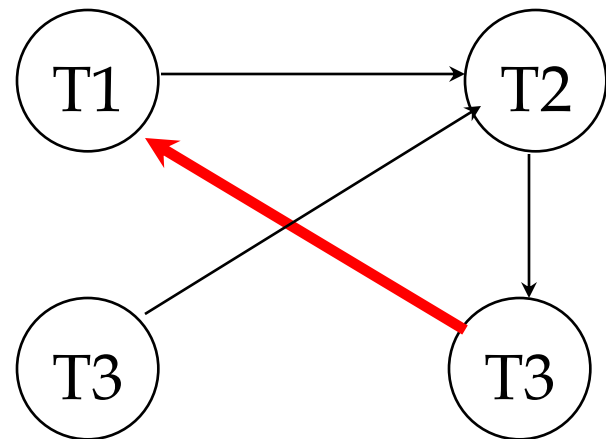
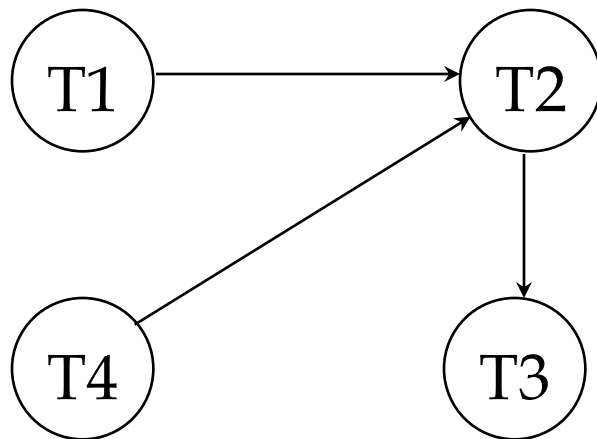
Deadlock Detection

- Create a **waits-for graph**:
 - Nodes are transactions
 - There is an edge from T_i to T_j if T_i is waiting for T_j to release a lock
- Periodically check for cycles in the waits-for graph
- If deadlock occurs, pick a *victim* transaction and abort.

Deadlock Detection (contd.)

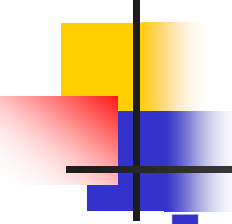
Example:

T1: S(A), R(A), S(B)
T2: X(B), W(B) X(C)
T3: S(C), R(C) X(A)
T4: X(B)



Dynamic Databases & Phantoms

- If we relax the assumption that the DB is a fixed collection of objects, even Strict 2PL will not assure serializability:
 - T1 locks all pages containing sailor records with *rating* = 1, and finds oldest sailor (say, *age* = 71).
 - Next, T2 inserts a new sailor; *rating* = 1, *age* = 96.
 - T2 also deletes oldest sailor with *rating* = 2 (and, say, *age* = 80), and commits.
 - T1 now locks all pages containing sailor records with *rating* = 2, and finds oldest (say, *age* = 63).
- No consistent DB state where T1 is “correct”!



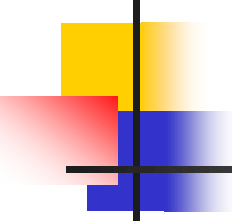
The Problem

- T1 implicitly assumes that it has locked the set of all sailor records with *rating* = 1.
 - Assumption only holds if no sailor records are added while T1 is executing!
 - Need some mechanism to enforce this assumption. (Predicate locking)
- Example shows that conflict serializability guarantees serializability only if the set of objects is fixed!

Index Locking

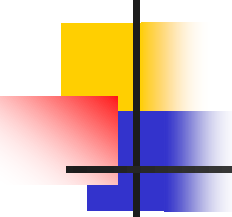


- If there is a dense index on the *rating* field using Alternative (2), T1 should lock the index page containing the data entries with *rating* = 1.
- If there are no records with *rating* = 1, T1 must lock the index page where such a data entry *would* be, if it existed!
- If there is no suitable index, T1 must lock all pages, and lock the file/table to prevent new pages from being added, to ensure that no new records with *rating* = 1 are added.



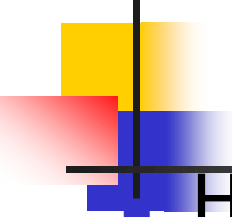
Predicate Locking

- Grant lock on all records that satisfy some logical predicate, e.g. *age > 2*salary*.
- Index locking is a special case of predicate locking for which an index supports efficient implementation of the predicate lock.
 - What is the predicate in the sailor example?
- In general, predicate locking has a lot of locking overhead.



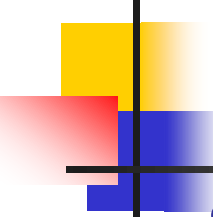
Locking in B+ Trees

- How can we efficiently lock a particular leaf node?
- One solution: Ignore the tree structure, just lock pages while traversing the tree, following 2PL.
- This has terrible performance!
 - Root node (and many higher level nodes) become bottlenecks because every tree access begins at the root.



Two Useful Observations

- Higher levels of the tree only direct searches for leaf pages.
- For inserts, a node on a path from root to modified leaf must be locked (in X mode, of course), only if a split can propagate up to it from the modified leaf. (Similar point holds w.r.t. deletes.)
- We can exploit these observations to design efficient locking protocols that guarantee serializability *even though they violate 2PL.*



A Simple Tree Locking Algorithm

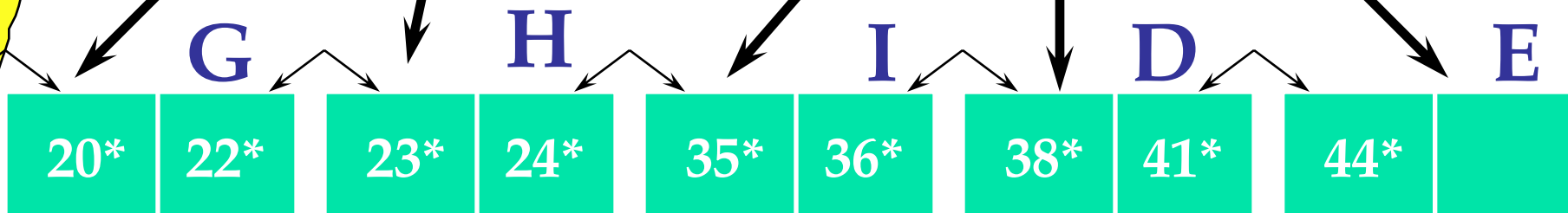
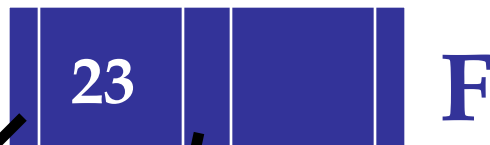
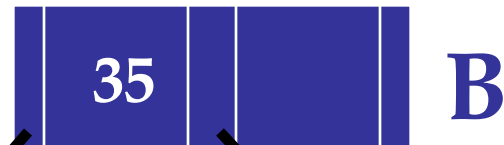
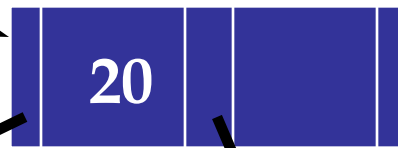
- **Search:** Start at root and go down; repeatedly, S lock child then unlock parent.
- **Insert/Delete:** Start at root and go down, obtaining X locks as needed. Once child is locked, check if it is safe:
 - If child is safe, release all locks on ancestors.
- **Safe node:** Node such that changes will not propagate up beyond this node.
 - Inserts: Node is not full.
 - Deletes: Node is not half-empty.

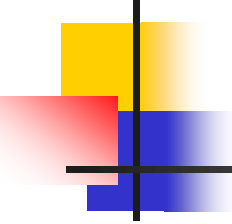
ROOT

Example

Do:

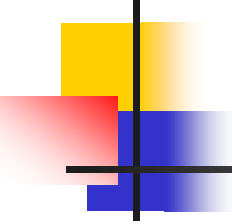
- 1) Search 38*
- 2) Delete 38*
- 3) Insert 45*
- 4) Insert 25*





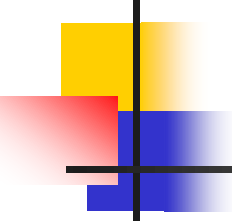
Lock Management...

- Lock and unlock requests are handled by the **lock manager**.
- Lock manager maintains a **lock table**
- **Lock table entry** for an object:
 - Number of transactions currently holding a lock
 - Type of lock held (shared or exclusive)
 - Pointer to queue of lock requests
- Locking and unlocking have to be atomic operations
- Lock upgrade: transaction that holds a shared lock can be upgraded to hold an exclusive lock



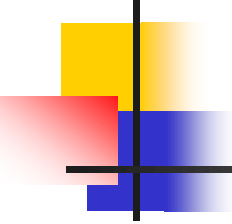
Lock Management...(contd.)

- DBMS also maintains a **transaction table** which contains a pointer to a list of locks held by the transactions
- Transaction requests a lock on an object
 - If shared lock is needed and the request queue is empty and object does not have exclusive lock, then grant & update lock entry
 - If an exclusive lock is needed and the request queue is empty and object is not in shared/exclusive mode, grant & update lock entry
 - Otherwise, add to the request to the request queue



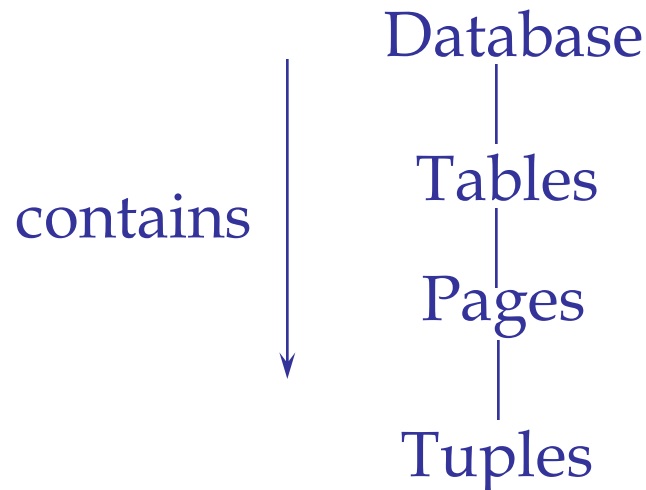
Multiple-Granularity Locking...

- Lock manager finds it difficult to manage locks
- Multiple Granularity: Files, pages, records etc.
 - An X-lock on page means that X-lock of file is not possible
 - Before granting an X-lock on file, need to scan all pages to verify no S or X locks on any page
- Solution: Use multiple-granularity locking scheme



Multiple-Granularity Locking... (contd.)

- Data “containers” are nested:



Solution: New Lock Modes, Protocol

- Allow Xacts to lock at each level, but with a special protocol using new “intention” locks:

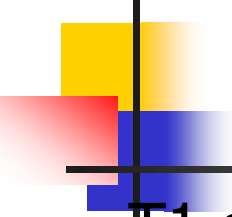
- v Before locking an item, Xact must set “intention locks” on all its ancestors.
- v For unlock, go from specific to general (i.e., bottom-up).
- v **SIX mode**: Like S & IX at the same time.

	--	IS	IX	S	X
--	✓	✓	✓	✓	✓
IS	✓	✓	✓	✓	
IX	✓	✓	✓		
S	✓	✓		✓	
X	✓				

Multiple Granularity Lock Protocol

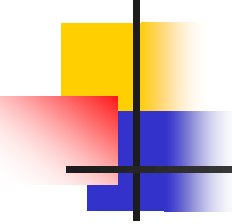
- Each Xact starts from the root of the hierarchy.
- To get S or IS lock on a node, must hold IS or IX on parent node.
 - What if Xact holds SIX on parent? S on parent?
- To get X or IX or SIX on a node, must hold IX or SIX on parent node.
- Must release locks in bottom-up order.

Protocol is correct in that it is equivalent to directly setting locks at the leaf levels of the hierarchy.



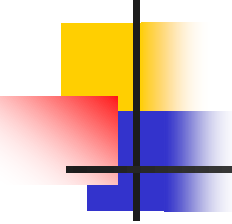
Examples

- T1 scans R, and updates a few tuples:
 - T1 gets an SIX lock on R, then repeatedly gets an S lock on tuples of R, and occasionally upgrades to X on the tuples.
- T2 uses an index to read only part of R:
 - T2 gets an IS lock on R, and repeatedly gets an S lock on tuples of R.
- T3 reads all of R:
 - T3 gets an S lock on R.
 - OR, T3 could behave like T2; can use **lock escalation** to decide which.



Transactions in SQL...

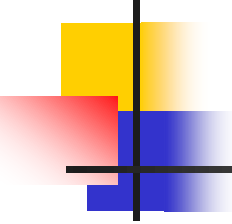
- Access Level: READ ONLY, READ WRITE
- Isolation Level:
 - SERIALIZABLE (Strict 2PL + Index Locking)
 - REPEATABLE READ (Strict 2PL)
 - Phantoms possible
 - READ COMMITTED (S-locks are released b/f end)
 - Phantoms+ Unrepeatable reads possible
 - READ UNCOMMITTED (can only be used with READ ONLY transactions)
 - Phantoms + Unrepeatable reads + Dirty reads possible



Other Concurrency Control Protocols...

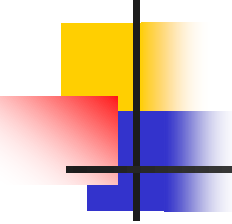
- Optimistic Concurrency Control Protocol
- Timestamp-Based Concurrency Control Protocol
- Multiversion Concurrency Control Protocol

Etc.



Summary

- Overheads of lock-based CC protocols
 - Deadlocks
 - Prevention and Detection
 - Phantoms
 - Predicate and Index locking
 - Handling locks
 - Multiple Granularity Locking
- Controlling Isolation level using SQL



Summary (contd.)

- Concurrency control and recovery are among the most important functions provided by a DBMS.
- Users need not worry about concurrency.
 - System automatically inserts lock/unlock requests and schedules actions of different Xacts in such a way as to ensure that the resulting execution is equivalent to executing the Xacts one after the other in some order.